

Docker, Docker-compose e K8s

Autor: Albert Santos Soares
Juan Pablo Barros Carvalho
Matheus Espírito Santo dos Santos
Rafael Pires Araújo

Agenda

O objetivo desta apresentação é mostrar a evolução da infraestrutura da MecâniQA Tech: de um servidor instável até a orquestração com containers.

1. Contexto:

Infraestrutura instável
exige isolamento dos
serviços

2. Docker:

Dockerfile isolado para API
Java, MySQL e Redis

3. Compose:

Orquestração local com
rede interna e volumes

4. Kubernetes (K8s):

Deployments e Services
com auto-healing

5. Conclusão:

Resultados obtidos e
próximos passos

Contexto do Projeto



- A MecâniQA Tech é uma startup SaaS de diagnóstico automotivo; a API Java roda em um servidor legado que sobrecarrega o MySQL e o Redis
- Desafio do Encontro 1: isolar cada serviço em containers, migrando de uma infraestrutura instável para uma fundação resiliente

Encontro 1 – Dockerfile



Decisões da equipe:

- eclipse-temurin:17-jdk-alpine
Imagem leve: compartilha o kernel do host (sem VM completa)
- EXPOSE 8080 no Dockerfile
-p 8080:8080 publica a porta da API no host
- ENTRYPOINT ["java", "-jar", "app.jar"] mantém o container ativo em primeiro plano

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

PS C:\mecanika\api> docker build -t mecanika-api .
[+] Building 12.4s (8/8) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> [internal] load .dockerignore                                                  0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk-alpine  1.2s
=> [internal] load build context                                                  0.3s
=> [1/4] FROM docker.io/library/eclipse-temurin:17-jdk-alpine                  8.7s
=> [2/4] WORKDIR /app                                                            0.1s
=> [3/4] COPY app.jar app.jar                                                    0.2s
=> [4/4] EXPOSE 8080                                                             0.1s
=> exporting to image                                                            0.2s
=> => writing image sha256:7f3c2a6e9e4b...                                       0.0s
=> => naming to docker.io/library/mecanika-api                                   0.0s

PS C:\mecanika\api> docker run -d -p 8080:8080 --name mecanika-api mecanika-api
a3f6e2c9b4d7e8f1c2a6b9d3e7f5c1a0b8c6d4e2f1a9b7c6d5e4f3a2b1c0d9e8f7
PS C:\mecanika\api>
```

Encontro 2 – Docker Compose



Decisões da equipe:

- docker-compose.yml integra os builds da API Java, do MySQL e do Redis
- DNS interno: a API acessa o MySQL por "db" e o Redis por "redis", sem IPs fixos
- Volumes garantem a persistência dos dados mesmo após reiniciar os containers

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

PS C:\mecanika> docker-compose up -d
[+] Running 4/4
= ✓ Network mecanika_default      Created          0.1s
= ✓ Volume "mecanika_mysql-data"  Created          0.0s
= ✓ Volume "mecanika_redis-data"  Created          0.0s
= ✓ Container mecanika-db         Started          1.2s
= ✓ Container mecanika-redis      Started          1.1s
= ✓ Container mecanika-api        Started          1.5s

PS C:\mecanika> docker-compose ps
Name                Command                                State              Ports
-----
mecanika-api        java -jar app.jar                     Up (healthy)       0.0.0.0:8080->8080/tcp
mecanika-db         docker-entrypoint.sh mysqld           Up (healthy)       3306/tcp
mecanika-redis      redis-server                           Up (healthy)       6379/tcp

PS C:\mecanika>
```

Encontro 3 – Kubernetes (K8s)

Decisões da equipe:

- O Deployments mantêm o estado desejado da aplicação e recriam Pods automaticamente em caso de falha
- O Services ClusterIP expõem MySQL e Redis internamente; LoadBalancer/NodePort expõem a API às oficinas clientes.
- O Terraform foi definido como próximo passo para o provisionamento da infraestrutura como código.

```
Context: kind-mecanica [R]
Cluster: kind-mecanica
User: kind-mecanica
K9s Rev: v0.51.0
K8s Rev: v1.34.0
CPU: n/a
MEM: n/a

<0> all      <d> Describe
<1> mecanica <?> Help
<2> default  <shift-j> Jump Owner
               <l> Logs
               <p> Logs Previous
               <shift-f> Port-Forward
```

NAME↑	PF	READY	STATUS	RESTARTS	IP	NODE	AGE
api-df7f9b654-kjbp9	●	1/1	Running	0	10.244.0.12	mecanica-control-plane	8m38s
mysql-db74b55d9-mmmn2	●	1/1	Running	0	10.244.0.11	mecanica-control-plane	10m
redis-7557ffd565-b4sr7	●	1/1	Running	0	10.244.0.10	mecanica-control-plane	10m

Considerações finais



- Deployments mantêm o estado desejado e recriam Pods automaticamente em caso de falha
- Services ClusterIP expõem MySQL e Redis internamente; LoadBalancer/NodePort expõem a API às oficinas clientes
- Próximo passo: provisionamento da infraestrutura como código com Terraform

Referências



- Caderno de Guias – OAT 1: Docker, Docker-Compose e K8s (UNEX, 2026)
- MecâniQA Automotive Tech – Programa de Trainee 2026.2, Contexto Inicial do Produto
- Documentação oficial: Docker, Docker Compose e Kubernetes